# Spatial Gesture Control: Controlling a Laptop with Head Orientation and Smartwatch Pinch Gestures

Vaibhav Bisht
*[Department, Institution]*
*[City, Country]*
vaibhavbisht78@gmail.com

***Abstract*—We present a system that lets a person control a laptop without touching it: they look at a calibrated screen location, pinch to grab a virtual volume knob, turn the wrist to change the volume and pinch again to release. Head orientation comes from consumer Sony headphones. Wrist motion and raw photoplethysmography (PPG) come from a Samsung Galaxy Watch over Bluetooth Low Energy, and all inference runs on the laptop behind a fail-closed safety chain. Because the watch cannot see the fingers, we recorded wrist signals with camera hand landmarks as timing ground truth and evaluated detectors as continuous event detectors, counting caught pinches against false activations per hour of no-gesture time. Wrist PPG alone generalised poorly to a new day (AUC 0.60–0.64). Fusing the inertial sensors raised worst-split AUC from 0.789 to 0.837, and a three-network 1-D convolutional model on nine raw channels caught about two thirds of pinches on unseen sessions, against 10–28% for tree ensembles, firing about 0.4 s sooner. We report the system design, the data pipeline, the results and the evaluation pitfalls that inflated earlier results.**

***Index Terms*—gesture recognition, smartwatch, photoplethysmography, inertial sensing, head tracking, human–computer interaction, wearable computing**

## I. Introduction

Laptops are still controlled through a keyboard, a pointer or a touch surface, even though many people now wear sensors on the head and wrist all day. Noise-cancelling headphones with motion sensors know where the head points, and a smartwatch measures wrist motion and blood volume. This paper asks whether those everyday wearables can replace the keyboard for a simple spatial control: a virtual volume knob that the user looks at, grabs with a finger pinch, turns with the wrist and releases with a second pinch.

The interaction is easy to describe and hard to build. A wrist-worn watch cannot see the fingers, so a pinch must be inferred from small changes in the optical photoplethysmography (PPG) signal and in the motion sensors. Everyday movement such as typing, reaching or scratching produces similar signals, and both signals shift between days and strap fits. The three devices keep separate clocks, and the system changes a real operating-system setting, so errors must never escalate.

We built and evaluated such a system over three and a half months (27 July to 7 November 2026). Our contributions are:

- A cross-platform desktop coordinator that fuses Sony headset orientation with Samsung Galaxy Watch motion and raw PPG, and drives system volume through a recipe engine with fail-closed safety.
- A data pipeline that records watch signals with camera hand landmarks as timing ground truth, and trains one binary detector per gesture label.
- An empirical study showing that wrist PPG alone generalises poorly to new days (AUC 0.60–0.64), that adding the inertial measurement unit (IMU) raises worst-split AUC from 0.789 to 0.837, and that a small 1-D convolutional network (CNN) on nine raw channels catches about two thirds of pinches on unseen sessions, against 10–28% for tree ensembles.
- An event-based evaluation standard (catches, repeats, late firings and false activations per hour of no-gesture time) applied identically in training and runtime, and a record of the evaluation pitfalls that inflated earlier results.

## II. Related Work

**Wrist PPG gestures.** FinDroidHR classified a vocabulary including pinch from a commercial smartwatch's optical sensor [1], and fSense estimated gesture force from smartwatch PPG [2]. Custom hardware has recognised finger-level gestures from two green sensors [3], simple hand gestures [4] and gestures from four spatial sensor sites using principal component analysis [5]. Robustness studies show that sensor shift degrades PPG gesture recognition [6] and that gesture and force level can be classified together [7]. Several headline results depend on other modalities: Eddy et al. segment PPG gestures with accelerometer boundaries [8], Zhao et al.'s sign-language result fuses motion sensors [9], and OpenWatch reports PPG as an addition to IMU (F1 of 90% on unseen people, with PPG adding 12.5 points) [10]. Multimodal wristbands [11], [12], datasets [13] and force estimation [14] complete the picture; reviews cover wearable gesture sensing [15] and wearable PPG more broadly [16]. Most published scores evaluate prompted, pre-segmented clips, often within one user. None establishes a continuously running, low-false-activation pinch detector from stock-watch PPG.

**Motion-based pinch detection.** Tsunoda et al. reached 94.7% for four gestures on unseen people using a 4,000 Hz accelerometer on a modified kernel, with accuracy falling sharply below 200 Hz [17]. ViWatch reached about 97% over a week at 100 Hz, but 45% on an unseen person without per-user adaptation [18], and Kim et al. detected 95% of

TABLE I
DEVICES AND DATA RATES

| Device | Data used | Rate |
|---|---|---|
| Sony headset | Orientation quaternion | ≈25 Hz |
| Galaxy Watch4+ | Rot. vector, lin. accel., gyro | 50 Hz |
| | Raw PPG green, red, IR | 25 Hz |
| | Crown button, wear state | On change |
| Webcam | 21 hand landmarks (labels only) | 23–30 fps |

deliberate pinches with a threshold on the high-passed 100 Hz signal [19]. On the watch used here, the raw stream available to applications is 50–100 Hz for motion and 25 Hz for PPG, so the high-rate route is closed.

**Robustness methods.** We screened group distributionally robust optimisation [20], domain-adversarial training [21], Deep CORAL [22], MiniRocket [23], invariant risk minimisation [24] and MixStyle [25] for cross-session robustness of PPG detectors.

## III. System Overview

Fig. 1 shows the architecture. The headset and watch only sense; every decision runs on the laptop. The desktop application is built with Tauri 2 [26], a Rust backend and a React frontend. Head orientation comes from the open-source `sony-head-tracker` [27], embedded in-process on macOS and Windows and run as a UDP bridge on Linux. The Wear OS watch application streams motion and raw PPG, read through Samsung's Health Sensor SDK [28], to the desktop over Bluetooth Low Energy (BLE) by default, using `btleplug` [29] on the desktop, or over a Wi-Fi WebSocket. A webcam running MediaPipe hand landmarks [30] is used only for labelling. Training runs offline in Python; inference runs in-process in Rust with no Python at runtime.

The target interaction has five steps. (1) The user looks at a calibrated location (top right by default) for a dwell time of 400 ms, and a transparent, non-focusable knob appears. (2) A *pinch in* grabs the knob and records the wrist pose and current volume. (3) Wrist roll changes volume relative to that pose. (4) A *pinch out* releases, keeping the volume, while the knob stays ready. (5) Looking away hides the knob; Escape cancels everything.

## IV. Head Targeting and Wrist Control

### A. Calibration and Dwell

The user faces the screen centre and then each named location while the system stores a unit quaternion per target. At runtime the nearest target is found by angular distance rather than separate yaw and pitch limits:

$$\theta = 2\arccos\big(|q_{\mathrm{cur}} \cdot q_{\mathrm{tgt}}|\big), \tag{1}$$

where the absolute value identifies $q$ and $-q$. A target activates when $\theta$ stays below an adjustable threshold for the dwell time. A change in the headset's reset counter means its reference frame moved, which invalidates the calibration.

TABLE II
VOLUME FILTERS AND LIMITS

| Stage | Rule |
|---|---|
| Velocity outlier | Freeze target above max. angular velocity |
| Jitter rejection | Accept target changes $\geq 0.5$ points |
| Easing | Exponential, 150 ms time constant |
| Slew limit | $\leq 30$ points/s, independent of input rate |
| Write pacing | $\geq 100$ ms apart, writer thread |
| Interaction epoch | Drop writes from a released grab |
| Clamp | 0–100 |

### B. Wrist-to-Volume Mapping

At grab time the watch orientation $Q_s$ is stored, and each new sample gives the relative rotation

$$Q_{\mathrm{rel}} = Q_s^{-1} Q_{\mathrm{cur}}, \tag{2}$$

from which the twist about the forearm axis is extracted. The volume target is absolute from the grab pose, so errors do not accumulate. The final mapping uses a $1^\circ$ dead zone and about one volume point per $3^\circ$, with a sensitivity setting and a direction switch. Table II lists the filters. On macOS each native volume call takes 130–190 ms, so writes are applied by a dedicated thread that always takes the newest target, keeping the sensor event loop free.

## V. Watch Sensing Pipeline

The watch sends JSON envelopes (type, version, device id, sequence, timestamp, payload) over either transport, sharing one decoder. Over BLE the watch is a GATT peripheral; the desktop is the central, assigns the device identity itself and requires both OS bonding and an explicit on-watch trust approval. Motion travels in 100 ms batches and PPG at each sensor flush (default 10 Hz), about 30 BLE notifications per second in total.

Building this path exposed several faults. Fragments sized as $\mathrm{MTU}-3 = 514$ bytes exceeded the 512-byte attribute limit, so Android silently refused every PPG batch over BLE. An unbounded queue grew to seconds of delay during arm movement, when the radio path weakens, and was then discarded whole. It was replaced by a half-second queue that drops the oldest whole messages. Sending each motion reading separately produced about 61 notifications per second, which saturated the link; batching motion and PPG as separate streams halved this. After these fixes, a 15-minute log showed 20 dropped messages (about 2 s of data), almost all just after reconnects. Timestamps stay on the watch's monotonic clock; all freshness and safety decisions use the desktop's own clock.

## VI. Pinch Detection

### A. Data and Ground Truth

Recordings are immutable bundles of PPG, IMU and labels. Pinch is split into `pinch_in` (contact), `pinch_hold` and `pinch_out` (release). Fist, thumbs-up and three kinds of everyday noise (typing, mouse use and ordinary activity) are negatives. Early labels came from on-screen cues, and each

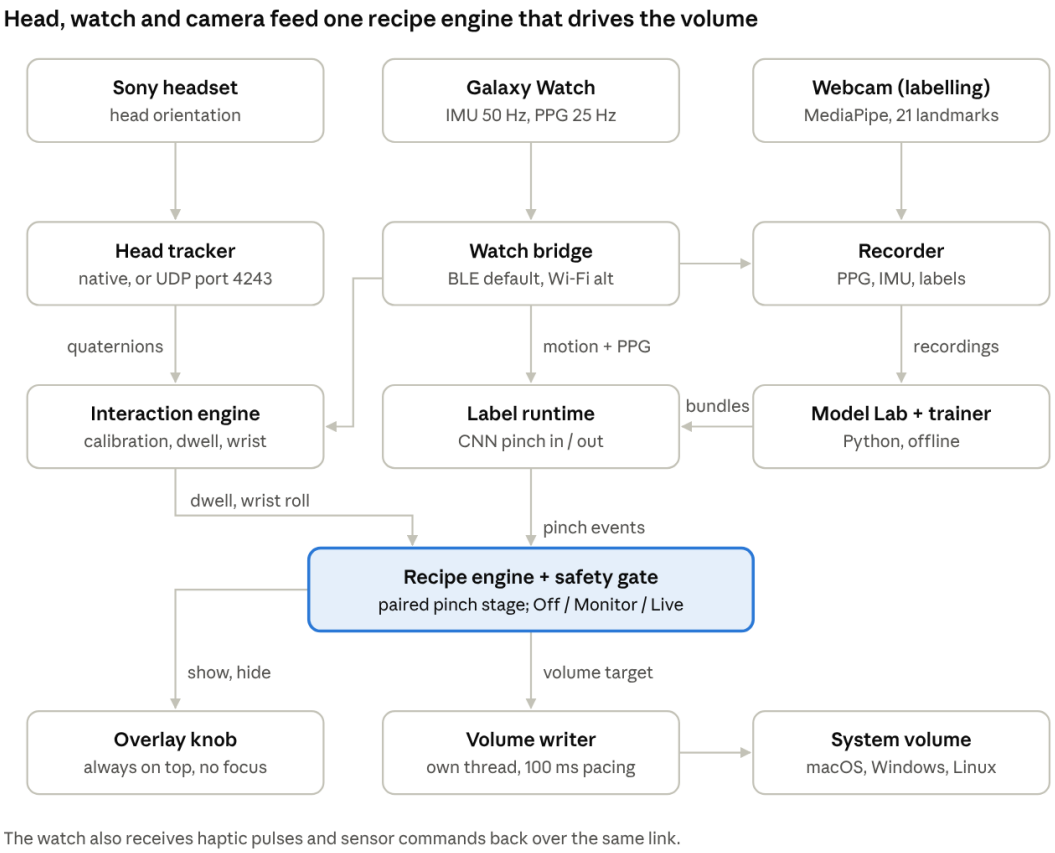



Fig. 1. System architecture. Head pose, wrist pose and pinch detections meet in one recipe engine, which alone may show the knob and drive the volume. The camera and trainer sit beside the live path.

recording had its own lag of up to 1.3 s. We therefore added camera ground truth. MediaPipe [30] finds 21 hand landmarks per frame in the application's web view, without network access; only landmarks are stored. Watch timestamps and browser arrival times are logged every 200 ms, and the smallest arrival-minus-watch difference estimates the clock offset. Pinch contact is marked from finger distance. A fixed threshold of 0.35 hand sizes missed many pinches, because apparent distance depends on hand orientation; judging each session against its own hand reduced false activations on pinch sessions from 110 to 20 per hour. All recordings are from one participant's left wrist.

### *B. Models*

Each label has its own binary model. Two kinds were developed (Table III).

**Trees.** 2,733 causal features: each PPG sample is z-scored against the preceding 1, 2 and 4 s, and 21 statistics of its slope, curvature and jerk are taken over the last 0.3, 0.6 and 1.3 s (1,701 features). The same statistics are taken for each motion axis and its magnitude (1,032 features). An ExtraTrees ensemble [31], [32] of 300 trees decides about 0.88 s after a gesture starts.

**CNN.** Nine channels (accelerometer, gyroscope, PPG green, red and infrared) on a 50 Hz grid. Each channel $x$ is normalised against its last 11 s with the median and median absolute deviation (MAD) and squashed:

$$\tilde{x}_t = \sigma\left(\frac{x_t - \mathrm{med}_{11\,\mathrm{s}}(x)}{\mathrm{MAD}_{11\,\mathrm{s}}(x)}\right), \tag{3}$$

and the last second (450 values) is the input. Three dilated convolutions (32, 32 and 64 channels, kernel 5, dilation 1, 2, 4) with batch normalisation feed per-channel max and mean pooling, a 32-unit dense layer with dropout and one output. Three networks trained from different seeds are averaged. Positives are samples 0.4–0.6 s after a start; negatives are samples with no start from 1.2 s before to 0.2 s after. Training uses Adam (learning rate 0.001, weight decay 0.0001), 12 epochs and batches of 256, in PyTorch [33]. The CNN decides about 0.5 s after a start.

### *C. Detection, Thresholds and Runtime*

A detection fires at the peak of the score once it exceeds the threshold, ends when the score falls 0.15 below it, and the next detection waits 1.5 s. The threshold is chosen on training recordings only. Five models, each trained without one fifth of them, score the held-out parts in replay, and the threshold that

TABLE III
THE TWO DETECTOR FAMILIES

| | Trees | CNN |
|---|---|---|
| Input | 2,733 features | $9 \times 50$ raw values |
| Model | 300 ExtraTrees | 3 dilated 1-D CNNs |
| Training moments | ≈1,000–1,300 | Tens of thousands |
| Decision delay | ≈0.88 s | ≈0.5 s |
| History needed | 5.6 s | 11 s |

catches the most starts within 30 false activations per hour is kept. Models ship as data-only bundles (tree node arrays or CNN weights) with a manifest. The SHA-256 hash and the input contract are verified on every load, and shared fixtures keep the Python and Rust input builders identical. A pinch-in detection is a one-shot event, and detector silence is not a release. A paired recipe stage therefore latches the grab on a fresh pinch-in while the head target is held, releases on pinch-out (release wins if both arrive together), and cancels on head exit, Escape, data loss, model change or leaving live mode.

## VII. Evaluation Methodology

Detectors are evaluated as continuous event detectors on whole recordings. Training and test recordings are always disjoint. Each emitted event is exactly one of: a *catch* (inside the match window after a labelled start), a *repeat* (another event during a caught gesture), *late*, or a *false activation*. The headline metrics are

$$\mathrm{TAR} = \frac{\#\,\text{caught starts}}{\#\,\text{labelled starts}}, \tag{4}$$

$$\mathrm{FAR} = \frac{\#\,\text{false activations}}{\text{hours with no gesture under way}}. \tag{5}$$

The FAR denominator is deliberately not total recording time. Pinch sessions are dense, so a rate per hour of recording hid a model that fired about 175 times per hour of no-gesture time. Study designs include recording-disjoint cross-validation judged by worst-split AUC, one-shot held-out days with the protocol frozen beforehand, leave-one-day-out tests, and replays with real motion bursts spliced in. Under a constant-rate Poisson assumption, zero false activations in $T$ hours bounds the rate at about $3/T$ per hour (95%), so short clean tests prove little.

## VIII. Results

### A. PPG Alone Is Not Enough

A PPG "step" at pinch onset appeared in every early session, but its rest controls had been chosen only by distance from gesture edges. A recheck found 84% of them (1,645 of 1,958) inside gestures, mostly in the middle of long holds. On a one-shot held-out day (14 October 2026; 208 pinch starts and labelled non-pinch activity), every PPG onset technique scored only 0.60–0.64 AUC against non-pinch moments. None separated fist starts from pinch starts (0.37–0.57), and none was usable as a continuous detector. The PPG-only tree model fired 19–25 times per minute on recordings containing no pinch.

### B. Adding the IMU

On a frozen benchmark of 37 recordings and 1,390 events in 20 recording-disjoint splits, fusing IMU with PPG raised worst-split AUC from 0.789 to 0.837 (better on 19 of 20 splits). It cut false activations on no-pinch recordings from 19.0 to 12.0 per minute at a threshold of 0.5. The original trigger fired at the first threshold crossing, often before the scoring window, and its lockout then blocked the correct firing. Firing at the score's peak added 4–12 recall points at the same false-activation budget, at a cost of 35–90 ms.

### C. CNN Against Trees

Table IV reports leave-one-day-out results on 372 camera-marked pinches. On new sessions the CNN caught about two thirds of pinches against 10–28% for trees, and fired about 0.4 s sooner. Applying a still-hand rule during training removed every moving moment, so the model never learned that movement is not a pinch. The best configuration trains without the rule and with everyday noise. The final pinch-in and pinch-out models caught 15 of 20 and 18 of 21 pinches on held-out sessions, and the participant reports that both work in the paired volume interaction.

### D. A Normalisation Blind Spot

The first CNN normalised each channel by a rolling 10 s mean and standard deviation. A burst of wrist motion raised the gyroscope deviation from about 0.03 to about 1 rad/s, shrinking a following pinch about thirty-fold until the burst left the window. No training pinch had seen this: the largest 1 s gyroscope RMS before any of 459 pinch starts was 0.62 rad/s, while everyday noise reached 14 rad/s. Table V shows that median/MAD scaling over 11 s removes the blind spot for bursts up to about 4 s.

### E. Other Variations

Retraining on the model's own false alarms lowered recall (27% against 31%) without reducing false activations. Self-supervised pretraining raised recall at its own threshold but almost quadrupled false activations on noise (27 against 7 per hour). A 2 s input window with a fourth convolution caught 49–51% against 45% at about 20 false activations per hour, but its automatically chosen threshold exceeded the budget on unseen noise. A curriculum (newest clean sessions, then older camera-labelled sessions, then noise) reached 78.9% recall for pinch-in at zero false activations on a small reserved validation pair. This is validation evidence only, not an independent test.

## IX. Safety Design

The system's external effects are bounded by a fail-closed chain. An ordering watermark, a quality gate, a window-length check, a fusion freshness check (orientation older than 500 ms is rejected) and a gesture policy each force a release when they reject data. A watchdog releases any grab whose model data has been silent for 750 ms. Inference runs in one of three modes: *Off*, *Monitor* (scores but never acts) or *Live* (acts only through explicit intent bindings). A saved Live setting is

TABLE IV
CNN VARIANTS, LEAVE ONE DAY OUT, 372 CAMERA-MARKED PINCHES

| Training | Caught | FAR noise (/h) | FAR pinch (/h) |
|---|---|---|---|
| October only, no noise (earlier)* | 204 (55%) | 143 | 23 |
| With still-hand rule | 146 (39%) | 23 | 20 |
| **Without the rule, with noise** | **161 (43%)** | **7** | 49† |
| Rule only at runtime | 130 (35%) | 3 | 16 |

FAR noise: false activations per hour on unseen everyday-noise sessions; FAR pinch: on pinch sessions. *Trained on most October pinches, so not unseen. †Dominated by one day; other days 0–28/h.

TABLE V
PINCH-IN DETECTION AFTER A SPLICED MOTION BURST

| Burst (ends 1 s before pinch) | Rolling | Median |
|---|---|---|
| None | 92% | 92% |
| 3–6 rad/s, 2 s | 0% | 93% |
| 2.5–6 rad/s, 4 s | 0% | 87% |
| 2–6 rad/s, 6 s | 0% | 58% |
| 2–6 rad/s, 8 s | 0% | 16% |

capped to Monitor at start-up. Grabs have one owner; the web view cannot show the overlay or inject gestures; the overlay lock is never held across a native call; and model bundles are re-verified on every load.

## X. LESSONS LEARNED

The largest errors came from evaluation rather than code. Rest controls that were really mid-gesture, explored recordings reused as tests, and false-activation rates over the wrong denominator each made detectors look better than they were. Ground truth mattered more than model choice: camera-timed, per-session labels removed more false activations than any architectural change. Training-time filters change what a model learns. Normalisation that looks harmless on clean data creates blind spots after realistic disturbances. On the engineering side, silently dropped BLE notifications, unbounded queues and blocking native calls each caused visible lag, and each was found by measuring before changing.

## XI. LIMITATIONS AND FUTURE WORK

All data come from one participant's left wrist, mostly at 50 Hz motion, and pinches made while moving are rare in the data. Sessions recorded on 14 October remain poorly detected (12–18%) by every model trained on other days. The best results are cross-validation and validation evidence; an untouched, multi-day test with refitted straps and long everyday negatives is still needed before any everyday false-activation claim. Physical validation on Windows and Linux, signed packaging, Wi-Fi authentication and a BLE battery trial remain open. Promising directions are per-user adaptation, which raised ViWatch from 45% to 97% [18], a longer input window with stricter threshold selection, and extending the same look-pinch-turn interaction to brightness and scrolling.

## XII. CONCLUSION

A headset and a smartwatch can control a laptop through a look-pinch-turn interaction. Head orientation selects the control, a CNN on watch motion and PPG detects pinch in and pinch out, and wrist roll sets the value, all behind a fail-closed safety chain. Wrist PPG alone was not sufficient on a stock watch. Motion fusion, camera-timed labels and event-based evaluation were what made the detector work.

## REFERENCES


[1] Zhang *et al.*, "FinDroidHR: Smartwatch gesture input with optical heartrate monitor," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, 2018.

[2] Buddhika *et al.*, "fSense: Unlocking the dimension of force for gestural interactions using smartwatch PPG sensor," in *Proc. Augmented Human International Conference*, 2019.

[3] Zhao *et al.*, "PPG-based finger-level gesture recognition leveraging wearables," in *Proc. IEEE INFOCOM*, 2018.

[4] Subramanian, Savur, and Sahin, "Using photoplethysmography for simple hand gesture recognition," in *Proc. IEEE System of Systems Engineering Conference (SoSE)*, 2020.

[5] Ruan *et al.*, "Principal component analysis of photoplethysmography signals for improved gesture recognition," *Frontiers in Neuroscience*, 2022.

[6] Fang *et al.*, "Exploration on the negative effects of sensor shifts in photoplethysmography-based gesture recognition and a solution based on transfer learning," *IEEE Transactions on Instrumentation and Measurement*, 2023.

[7] Li *et al.*, "Feasibility of wearable PPG for simultaneous hand gesture and force level classification," *IEEE Sensors Journal*, 2023.

[8] Eddy *et al.*, "Discrete gesture recognition using multimodal PPG, IMU, and single-channel EMG recorded at the wrist," *IEEE Sensors Letters*, 2024.

[9] Zhao *et al.*, "Towards low-cost sign language gesture recognition leveraging wearables," *IEEE Transactions on Mobile Computing*, 2021.

[10] Bonazzi *et al.*, "OpenWatch: A multimodal benchmark for hand gesture recognition on smartwatches," arXiv:2605.04791v2, 2026. [Online]. Available: https://arxiv.org/html/2605.04791v2

[11] Wang *et al.*, "A novel PPG-FMG-ACC wristband for hand gesture recognition," *IEEE Journal of Biomedical and Health Informatics*, 2022.

[12] [Author list to be completed], "Multi-modality gesture recognition for portable wearables: A high-integration system combining electromyography, inertial measurement unit, and photoplethysmography," *Engineering Applications of Artificial Intelligence*, 2026.

[13] Hauptmann *et al.*, "$\mu$DPad: A large-scale multimodal PPG and IMU dataset for wrist-worn microgesture recognition," in *ACM conference proceedings*, 2026.

[14] Hu *et al.*, "WatchForce: Wrist-worn PPG and IMU sensing for hand force estimation," *Computer Methods and Programs in Biomedicine*, 2026.

[15] Tchantchane *et al.*, "Review of hand gesture recognition systems based on noninvasive wearable sensors," *Advanced Intelligent Systems*, 2023.

[16] Charlton *et al.*, "The 2023 wearable photoplethysmography roadmap," *Physiological Measurement*, 2023.

[17] Tsunoda *et al.*, "Microgesture recognition with a high-rate smartwatch accelerometer," in *Proc. Int. Conf. Mobile and Ubiquitous Multimedia (MUM)*, 2024. [Online]. Available: https://www.iplab.cs.tsukuba.ac.jp/paper/international/tsunoda_MUM2024.pdf
[18] ViWatch authors, "ViWatch: Vibration-based gesture input on smartwatches," in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2023. [Online]. Available: https://www.cs.virginia.edu/~stankovic/psfiles/ViWatch_UIST_%20FINAL.pdf
[19] Kim *et al.*, "Pinch detection from a smartwatch accelerometer," in *Proc. ACM CHI Conference on Human Factors in Computing Systems*, 2025, arXiv:2503.02308. [Online]. Available: https://arxiv.org/abs/2503.02308
[20] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, "Distributionally robust neural networks for group shifts," arXiv:1911.08731, 2019.
[21] Ganin *et al.*, "Domain-adversarial training of neural networks," arXiv:1505.07818, 2015.
[22] B. Sun and K. Saenko, "Deep CORAL: Correlation alignment for deep domain adaptation," arXiv:1607.01719, 2016.
[23] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," arXiv:2012.08791, 2020.
[24] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, "Invariant risk minimization," arXiv:1907.02893, 2019.
[25] K. Zhou, Y. Yang, Y. Qiao, and T. Xiang, "Domain generalization with MixStyle," arXiv:2104.02008, 2021.
[26] Tauri Contributors, "Tauri 2.0," https://v2.tauri.app/, accessed 7 Nov. 2026.
[27] N. Slattery, "sony-head-tracker: Sony headphone motion sensors as a low-latency head tracker, release v2.2.0," https://github.com/NicholasSlattery/sony-head-tracker, accessed 7 Nov. 2026.
[28] Samsung Electronics, "Samsung health sensor SDK," https://developer.samsung.com/health/sensor/overview.html, accessed 7 Nov. 2026.
[29] deviceplug, "btleplug: Rust cross-platform host-side bluetooth LE access library," https://github.com/deviceplug/btleplug, accessed 7 Nov. 2026.
[30] Google, "MediaPipe hand landmarker task guide," https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, accessed 7 Nov. 2026.
[31] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
[32] scikit-learn developers, "ExtraTreesClassifier, scikit-learn documentation," https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesClassifier.html, accessed 7 Nov. 2026.
[33] PyTorch Foundation, "PyTorch," https://pytorch.org/, accessed 7 Nov. 2026.